

Typical Acceptance理解

Thinking: 假设模型正在生成句子, 当前要预测"The weather today is"后面的词:

模型预测分布:

"sunny" → $p=0.35$, 信息量 $=-\log(0.35)=1.51$

"good" → $p=0.25$, 信息量 $=-\log(0.25)=2.00$

"nice" → $p=0.20$, 信息量 $=-\log(0.20)=2.32$

"cold" → $p=0.10$, 信息量 $=-\log(0.10)=3.32$

"banana" → $p=0.001$, 信息量 $=-\log(0.001)=9.97$

熵 $H = 2.1$ (当前分布的平均不确定性)

Typical Acceptance 的判断:

候选 token	信息量	与熵H的距离	判定
"sunny"	1.51	$ 1.51-2.1 =0.59$	接受 (接近熵)
"good"	2.00	$ 2.00-2.1 =0.10$	接受 (非常接近)
"nice"	2.32	$ 2.32-2.1 =0.22$	接受 (接近熵)
"cold"	3.32	$ 3.32-2.1 =1.22$	拒绝 (偏 surprising)
"banana"	9.97	$ 9.97-2.1 =7.87$	拒绝 (太 surprising)

温度高 → 接受阈值更宽松 → 更多候选被接受 → 解码更快, 但质量可能下降

温度低 → 接受阈值更严格 → 只接受最"典型"的 → 质量更高, 但速度慢

这张图片中提到的**“典型接受” (Typical Acceptance) **, 是推测解码中为了提高“草稿”接受率、同时保证生成文本自然度的一种极其聪明的高阶采样与验证策略。

它背后的核心数学理论来自于香农信息论 (Shannon Information Theory)。我来为你抽丝剥茧, 把这三个问题彻底讲透:

1. 图中的“熵 (Entropy)”是什么? 如何计算?

通俗理解:

在大模型预测下一个词时, “熵”代表了模型在当前语境下的**“迷茫程度”或“预期的平均惊喜度”

**。

- 如果模型非常确信下一个词是什么 (比如“我爱中”后面接“国”的概率是 99%), 那么此时的熵极低。
- 如果模型觉得后面接什么词都有可能 (比如“今天天气很”后面可以接好、差、热、冷), 各种词的的概率差不多, 此时的熵极高。

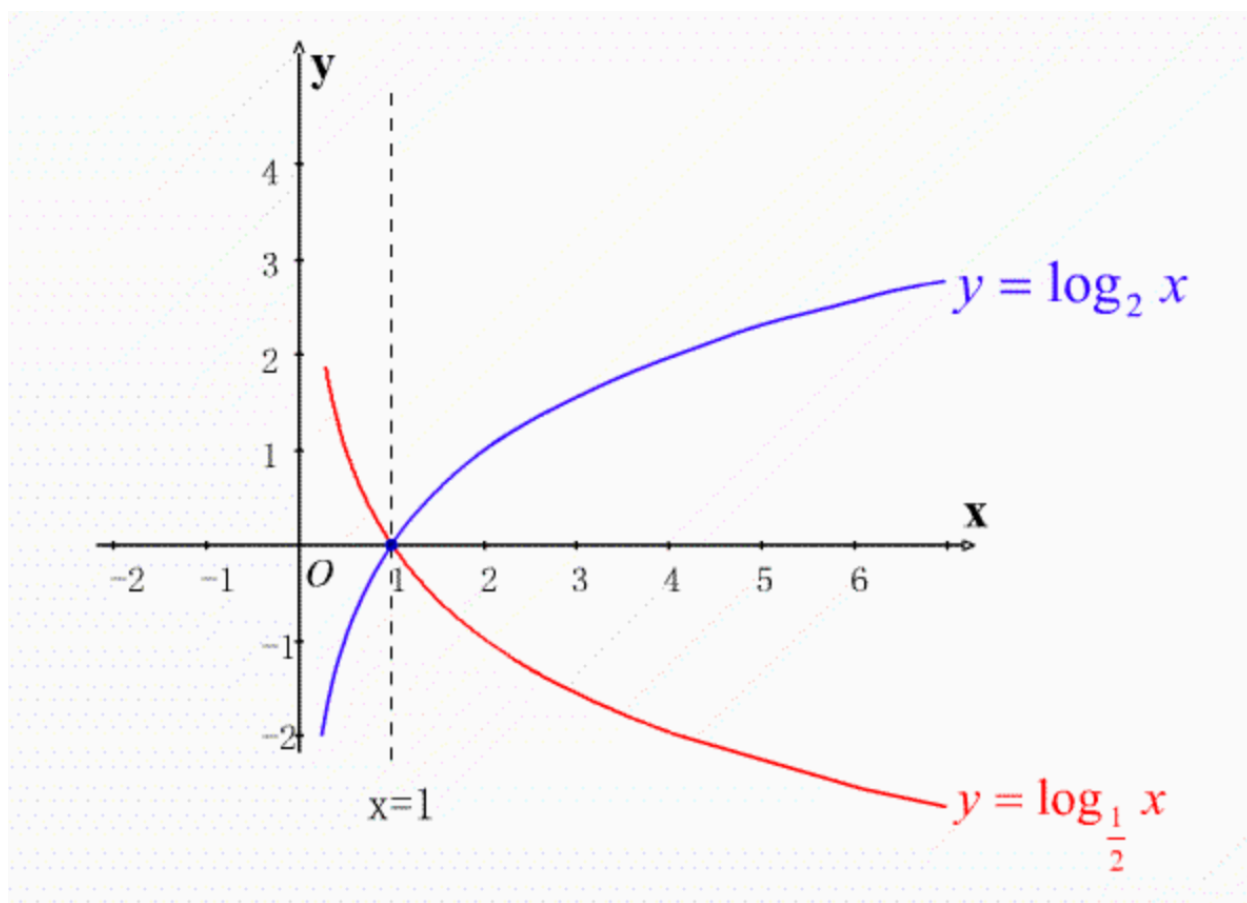
如何计算:

大模型 (Target Model) 在当前位置会对词表中的所有词 (假设有 N 个词) 算出一个概率分布 P 。熵 $H(P)$ 的计算公式就是所有词概率与其对数乘积的负和:

$$H(P) = - \sum_{x \in V} P(x) \log P(x)$$

(其中 V 是整个词表, $P(x)$ 是大模型预测词 x 出现的概率)

对数曲线



这个公式描述的是 信息熵 (**Entropy**), 在大型语言模型中, 它衡量了模型预测下一个词时的“不确定性”。

为了让你直观理解, 我们假设一个极简的场景。

1. 设定场景

假设大模型的词表 V 里只有 3 个词: ["苹果", "香蕉", "西瓜"]。

当模型预测下一个词时, 它给出了一个概率分布 P :

- $P(\text{苹果}) = 0.5$
 - $P(\text{香蕉}) = 0.3$
 - $P(\text{西瓜}) = 0.2$
- (注意: 所有概率之和必须等于 1)

2. 带入公式计算

公式为: $H(P) = -\sum_{x \in V} P(x) \log P(x)$

(在计算机科学中, $\log$ 通常以 2 为底, 单位是比特 bit)

我们将三个词的数据分别带入并相加:

- 苹果: $0.5 \times \log_2(0.5) = 0.5 \times (-1) = -0.5$
- 香蕉: $0.3 \times \log_2(0.3) \approx 0.3 \times (-1.737) \approx -0.521$
- 西瓜: $0.2 \times \log_2(0.2) \approx 0.2 \times (-2.322) \approx -0.464$

求和:

$\sum = (-0.5) + (-0.521) + (-0.464) = -1.485$

取负号:

$H(P) = -(-1.485) = \mathbf{1.485}$

3. 这个结果说明了什么?

- 如果熵很高: 说明模型很“纠结”, 概率分布很均匀 (例如每个词都是 0.33), 它不知道该选哪个词。
- 如果熵很低: 说明模型很“确定”, 某个词的概率接近 1 (例如 $P(\text{苹果})=0.99$), 预测非常清晰。

总结计算步骤

1. 找出模型对每个词预测的概率值。
2. 计算每个概率的自信息量 (即 $-\log P(x)$)。
3. 用概率作为权重, 计算所有词的加权平均值。

在实际的大模型 (如 GPT-4) 中, 词表 V 通常有几万到十万个词, 计算过程完全一样, 只是项数更多。

你是在研究模型评估指标 (比如 Perplexity 困惑度) 吗? 那个指标本质上就是熵的指数形式。